

SMS Spam Classifier

May 2, 2025

Importing Libraries

```
[1]: import numpy as np
import pandas as pd
```

```
[2]: df1=pd.read_csv('Spam.csv',encoding='latin-1')
```

```
[3]: df1.head()
```

```
[3]:      v1                                v2 Unnamed: 2  \
0   ham  Go until jurong point, crazy.. Available only ...      NaN
1   ham                                Ok lar... Joking wif u oni...      NaN
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...      NaN
3   ham  U dun say so early hor... U c already then say...      NaN
4   ham  Nah I don't think he goes to usf, he lives aro...      NaN
```

```
      Unnamed: 3 Unnamed: 4
0           NaN           NaN
1           NaN           NaN
2           NaN           NaN
3           NaN           NaN
4           NaN           NaN
```

```
[4]: df1.shape
```

```
[4]: (5572, 5)
```

```
[5]: df1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5572 entries, 0 to 5571
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   v1              5572 non-null  object
1   v2              5572 non-null  object
2   Unnamed: 2      50 non-null    object
3   Unnamed: 3      12 non-null    object
4   Unnamed: 4      6 non-null     object
```

```
dtypes: object(5)
memory usage: 217.8+ KB
```

```
[6]: #drop last 3 cloumns
df1.drop(columns=['Unnamed: 2', 'Unnamed: 3', 'Unnamed: 4'], inplace=True)
```

```
[7]: df1.head()
```

```
[7]:      v1                                     v2
0  ham  Go until jurong point, crazy.. Available only ...
1  ham                                     Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3  ham  U dun say so early hor... U c already then say...
4  ham  Nah I don't think he goes to usf, he lives aro...
```

```
[8]: from sklearn.preprocessing import LabelEncoder
encoder=LabelEncoder()
```

```
[9]: #renaming the columns
df1.rename(columns={"v1": "target", "v2": "text"}, inplace=True)
```

```
[10]: df1['target']=encoder.fit_transform(df1['target'])
```

```
[11]: df1['target']
```

```
[11]: 0      0
1      0
2      1
3      0
4      0
..
5567   1
5568   0
5569   0
5570   0
5571   0
Name: target, Length: 5572, dtype: int32
```

```
[12]: df1
```

```
[12]:      target                                     text
0         0  Go until jurong point, crazy.. Available only ...
1         0                                     Ok lar... Joking wif u oni...
2         1  Free entry in 2 a wkly comp to win FA Cup fina...
3         0  U dun say so early hor... U c already then say...
4         0  Nah I don't think he goes to usf, he lives aro...
...      ...
5567      1  This is the 2nd time we have tried 2 contact u...
```

```

5568      0      Will Ì_ b going to esplanade fr home?
5569      0  Pity, * was in mood for that. So...any other s...
5570      0  The guy did some bitching but I acted like i'd...
5571      0      Rofl. Its true to its name

```

```
[5572 rows x 2 columns]
```

```
[13]: #missing values
df1.isnull().sum()
```

```
[13]: target      0
      text      0
      dtype: int64
```

```
[14]: #check for duplicates
df1.duplicated().sum()
```

```
[14]: 403
```

```
[15]: #remove duplicates
df1=df1.drop_duplicates(keep='first')
```

```
[16]: df1.duplicated().sum()
```

```
[16]: 0
```

```
[17]: df1.duplicated().sum()
```

```
[17]: 0
```

```
[18]: df1.head()
```

```
[18]:      target      text
0      0  Go until jurong point, crazy.. Available only ...
1      0      Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...

```

```
[19]: df1.shape
```

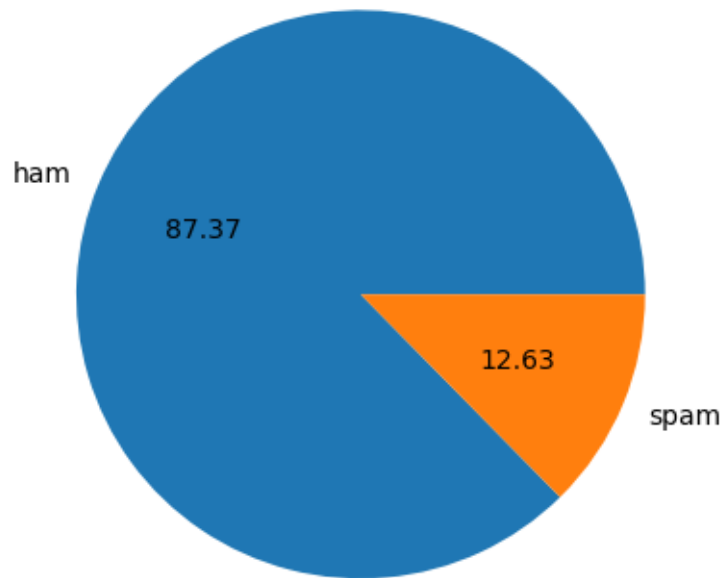
```
[19]: (5169, 2)
```

```
[20]: df1['target'].value_counts()
```

```
[20]: target
0      4516
1      653
```

Name: count, dtype: int64

```
[21]: import matplotlib.pyplot as plt
plt.pie(df1['target'].value_counts(), labels=['ham', 'spam'], autopct="%0.2f")
plt.show()
```



```
[22]: #data is imbalanced
import nltk
nltk.download('punkt')
```

```
[nltk_data] Downloading package punkt to
[nltk_data] C:\Users\Chinnu\AppData\Roaming\nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

[22]: True

```
[23]: df1['num_characters']=df1['text'].apply(len)
```

```
C:\Users\Chinnu\AppData\Local\Temp\ipykernel_3408\1998808907.py:1:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

See the caveats in the documentation: <https://pandas.pydata.org/pandas->

```
docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
df1['num_characters']=df1['text'].apply(len)
```

```
[24]: df1['num_characters']
```

```
[24]: 0      111
      1      29
      2     155
      3      49
      4      61
      ...
     5567    161
     5568     37
     5569     57
     5570    125
     5571     26
      Name: num_characters, Length: 5169, dtype: int64
```

```
[25]: df1.head()
```

```
[25]:   target      text  num_characters
0      0  Go until jurong point, crazy.. Available only ...      111
1      0              Ok lar... Joking wif u oni...       29
2      1  Free entry in 2 a wkly comp to win FA Cup fina...     155
3      0  U dun say so early hor... U c already then say...     49
4      0  Nah I don't think he goes to usf, he lives aro...     61
```

```
[26]: #num of words
df1['num_words']=df1['text'].apply(lambda x:len(nltk.word_tokenize(x)))
```

```
C:\Users\Chinnu\AppData\Local\Temp\ipykernel_3408\2555552905.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead
```

```
See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
df1['num_words']=df1['text'].apply(lambda x:len(nltk.word_tokenize(x)))
```

```
[27]: df1
```

```
[27]:   target      text \
0      0  Go until jurong point, crazy.. Available only ...
1      0              Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...
...    ...
```

```

5567      1  This is the 2nd time we have tried 2 contact u...
5568      0              Will I_ b going to esplanade fr home?
5569      0  Pity, * was in mood for that. So...any other s...
5570      0  The guy did some bitching but I acted like i'd...
5571      0              Rofl. Its true to its name

```

```

      num_characters  num_words
0              111         24
1              29          8
2             155         37
3              49         13
4              61         15
...
5567             161         35
5568             37          9
5569             57         15
5570            125         27
5571             26          7

```

[5169 rows x 4 columns]

```

[28]: #num of sentences
df1['num_sentences']=df1['text'].apply(lambda x:len(nltk.sent_tokenize(x)))

```

C:\Users\Chinnu\AppData\Local\Temp\ipykernel_3408\2149131253.py:2:

SettingWithCopyWarning:

A value is trying to be set on a copy of a slice from a DataFrame.

Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
df1['num_sentences']=df1['text'].apply(lambda x:len(nltk.sent_tokenize(x)))
```

```
[29]: df1
```

```

[29]:      target      text \
0      0  Go until jurong point, crazy.. Available only ...
1      0              Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...
...
5567      1  This is the 2nd time we have tried 2 contact u...
5568      0              Will I_ b going to esplanade fr home?
5569      0  Pity, * was in mood for that. So...any other s...
5570      0  The guy did some bitching but I acted like i'd...
5571      0              Rofl. Its true to its name

```

	num_characters	num_words	num_sentences
0	111	24	2
1	29	8	2
2	155	37	2
3	49	13	1
4	61	15	1
...	...	...	...
5567	161	35	4
5568	37	9	1
5569	57	15	2
5570	125	27	1
5571	26	7	2

[5169 rows x 5 columns]

```
[30]: df1[['num_characters', 'num_words', 'num_sentences']].describe()
```

```
[30]:
```

	num_characters	num_words	num_sentences
count	5169.000000	5169.000000	5169.000000
mean	78.977945	18.455794	1.965564
std	58.236293	13.324758	1.448541
min	2.000000	1.000000	1.000000
25%	36.000000	9.000000	1.000000
50%	60.000000	15.000000	1.000000
75%	117.000000	26.000000	2.000000
max	910.000000	220.000000	38.000000

```
[31]: #ham
df1[df1['target']==0][['num_characters', 'num_words', 'num_sentences']].describe()
```

```
[31]:
```

	num_characters	num_words	num_sentences
count	4516.000000	4516.000000	4516.000000
mean	70.459256	17.123782	1.820195
std	56.358207	13.493970	1.383657
min	2.000000	1.000000	1.000000
25%	34.000000	8.000000	1.000000
50%	52.000000	13.000000	1.000000
75%	90.000000	22.000000	2.000000
max	910.000000	220.000000	38.000000

```
[32]: #spam
df1[df1['target']==1][['num_characters', 'num_words', 'num_sentences']].describe()
```

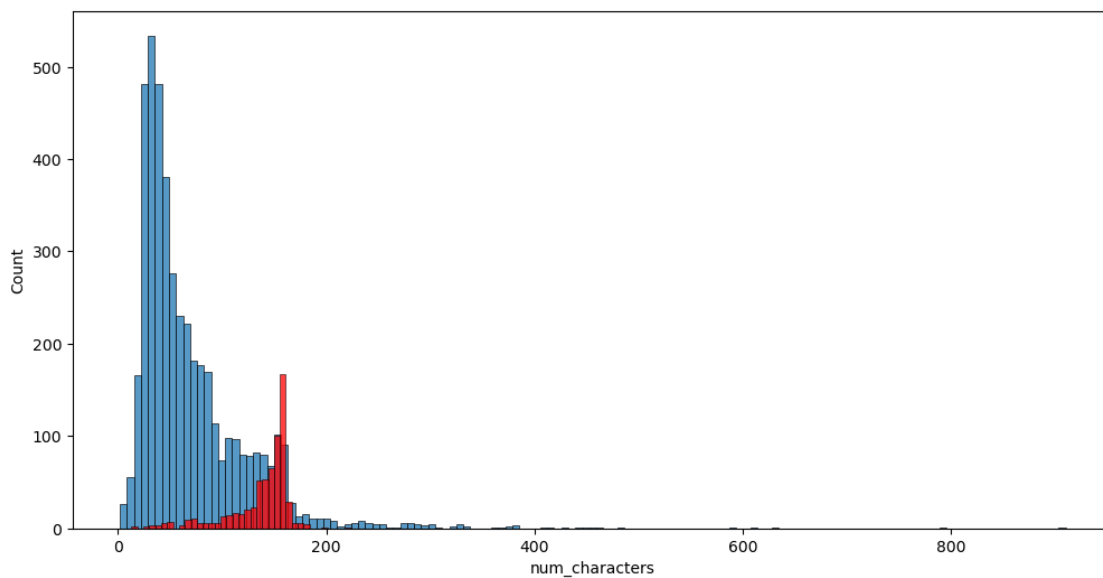
```
[32]:
```

	num_characters	num_words	num_sentences
count	653.000000	653.000000	653.000000
mean	137.891271	27.667688	2.970904
std	30.137753	7.008418	1.488425

min	13.000000	2.000000	1.000000
25%	132.000000	25.000000	2.000000
50%	149.000000	29.000000	3.000000
75%	157.000000	32.000000	4.000000
max	224.000000	46.000000	9.000000

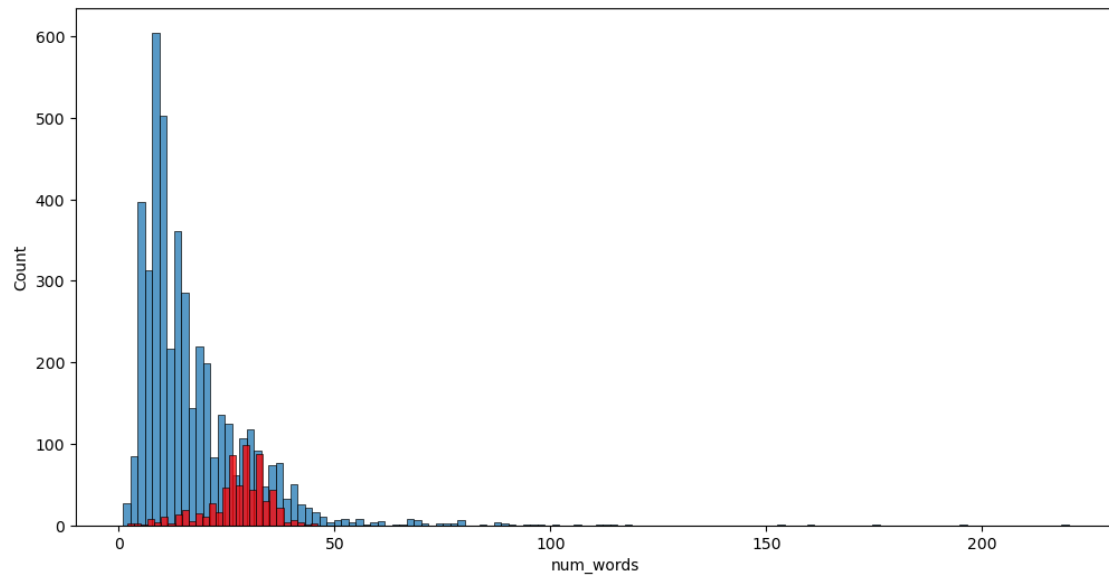
```
[33]: import seaborn as sns
plt.figure(figsize=(12,6))
sns.histplot(df1[df1['target']==0]['num_characters'])
sns.histplot(df1[df1['target']==1]['num_characters'],color='red')
```

[33]: <Axes: xlabel='num_characters', ylabel='Count'>



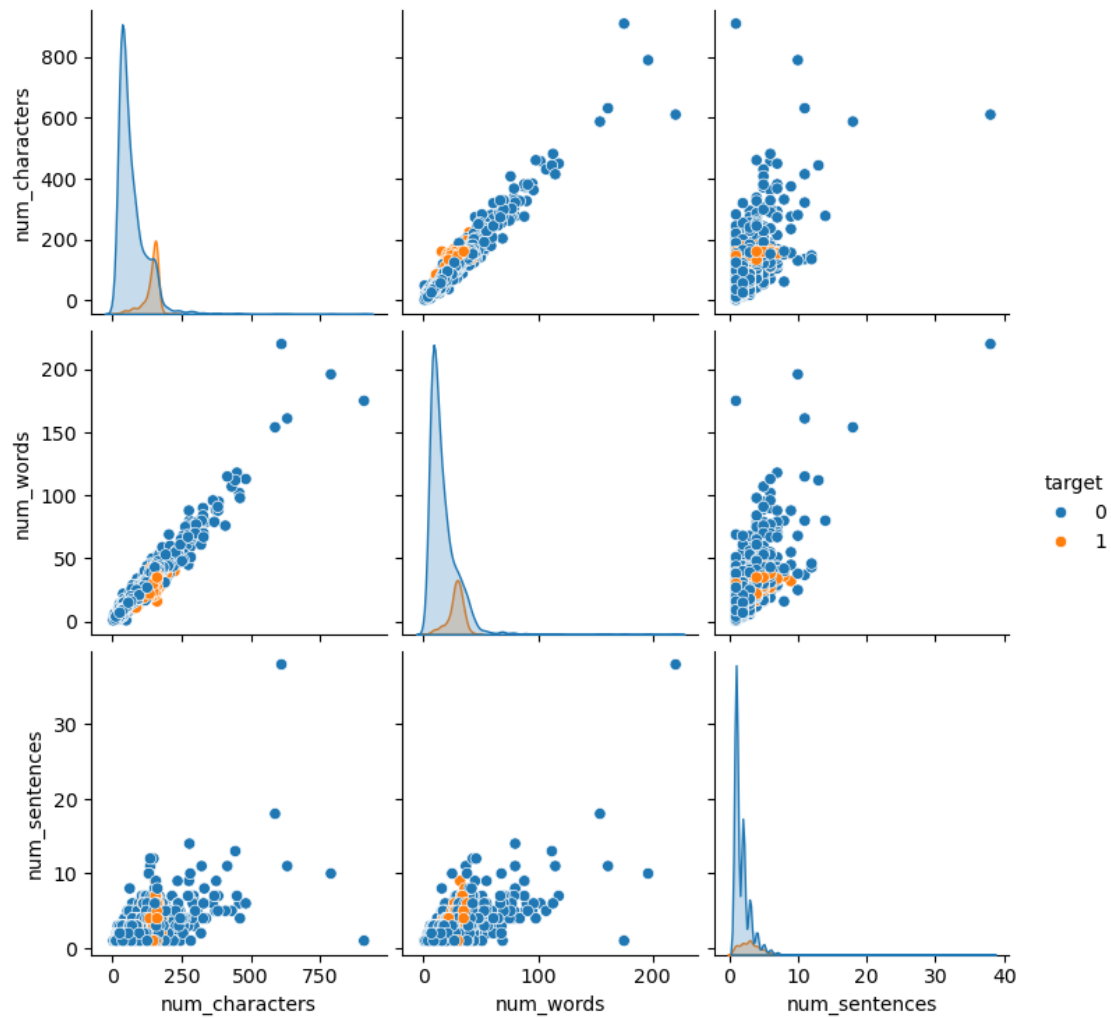
```
[34]: plt.figure(figsize=(12,6))
sns.histplot(df1[df1['target']==0]['num_words'])
sns.histplot(df1[df1['target']==1]['num_words'],color='red')
```

[34]: <Axes: xlabel='num_words', ylabel='Count'>



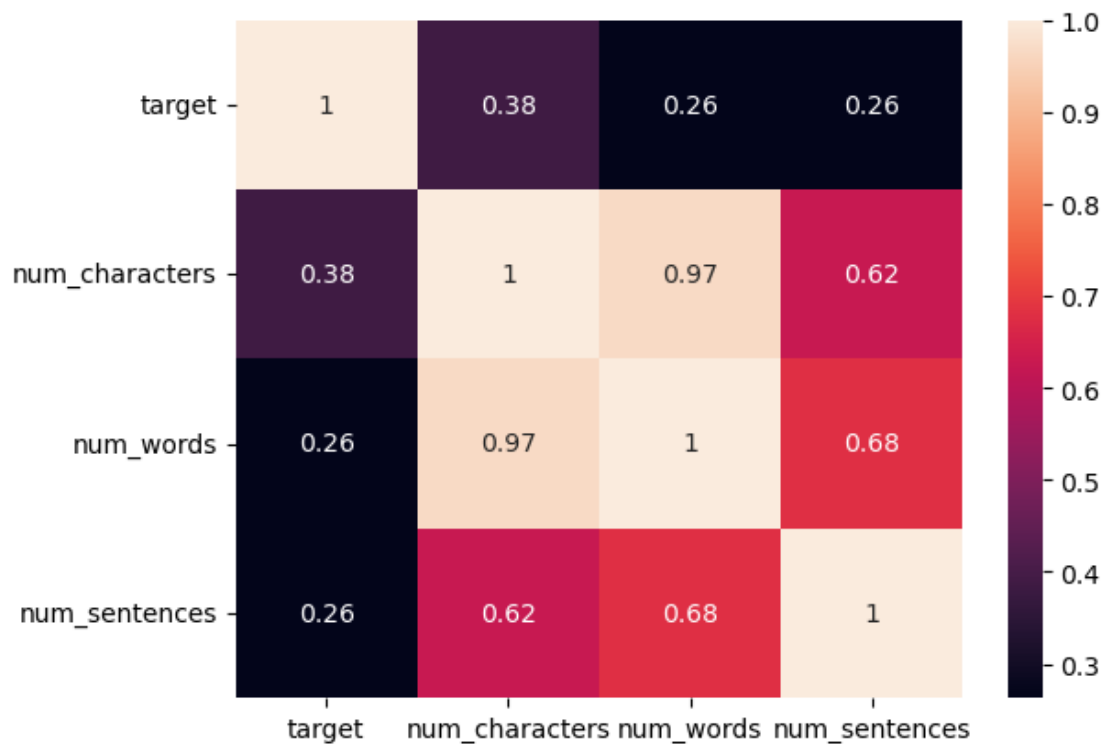
```
[35]: sns.pairplot(df1,hue='target')
```

```
[35]: <seaborn.axisgrid.PairGrid at 0x1cb94597470>
```



```
[36]: numeric_df = df1.select_dtypes(include='number')
      sns.heatmap(numeric_df.corr(),annot=True)
```

```
[36]: <Axes: >
```



Data Preprocessing

```
[49]: def transform_text(text):
    text=text.lower()
    text=nlk.word_tokenize(text)
    y=[]
    for i in text:
        if i.isalnum():
            y.append(i)

    text=y[:]
    y.clear()

    for i in text:
        if i not in stopwords.words('english') and i not in string.punctuation:
            y.append(i)

    text=y[:]
    y.clear()

    for i in text:
        y.append(ps.stem(i))
```

```
return " ".join(y)
```

```
[50]: from nltk.corpus import stopwords  
stopwords.words('english')
```

```
[50]: ['a',  
      'about',  
      'above',  
      'after',  
      'again',  
      'against',  
      'ain',  
      'all',  
      'am',  
      'an',  
      'and',  
      'any',  
      'are',  
      'aren',  
      "aren't",  
      'as',  
      'at',  
      'be',  
      'because',  
      'been',  
      'before',  
      'being',  
      'below',  
      'between',  
      'both',  
      'but',  
      'by',  
      'can',  
      'couldn',  
      "couldn't",  
      'd',  
      'did',  
      'didn',  
      "didn't",  
      'do',  
      'does',  
      'doesn',  
      "doesn't",  
      'doing',  
      'don',  
      "don't",  
      'down',
```

'during',
'each',
'few',
'for',
'from',
'further',
'had',
'hadn',
'hadn't',
'has',
'hasn',
'hasn't',
'have',
'haven',
'haven't',
'having',
'he',
'he'd',
'he'll',
'her',
'here',
'hers',
'herself',
'he's',
'him',
'himself',
'his',
'how',
'i',
'i'd',
'if',
'i'll',
'i'm',
'in',
'into',
'is',
'isn',
'isn't',
'it',
'it'd',
'it'll',
'it's',
'its',
'itself',
'i've',
'just',
'll',

'm',
'ma',
'me',
'mightn',
"mightn't",
'more',
'most',
'mustn',
"mustn't",
'my',
'myself',
'needn',
"needn't",
'no',
'nor',
'not',
'now',
'o',
'of',
'off',
'on',
'once',
'only',
'or',
'other',
'our',
'ours',
'ourselves',
'out',
'over',
'own',
're',
's',
'same',
'shan',
"shan't",
'she',
"she'd",
"she'll",
"she's",
'should',
'shouldn',
"shouldn't",
"should've",
'so',
'some',
'such',

't',
'than',
'that',
"that'll",
'the',
'their',
'theirs',
'them',
'themselves',
'then',
'there',
'these',
'they',
"they'd",
"they'll",
"they're",
"they've",
'this',
'those',
'through',
'to',
'too',
'under',
'until',
'up',
've',
'very',
'was',
'wasn',
"wasn't",
'we',
"we'd",
"we'll",
"we're",
'were',
'weren',
"weren't",
"we've",
'what',
'when',
'where',
'which',
'while',
'who',
'whom',
'why',
'will',

```
'with',
'won',
"won't",
'wouldn',
"wouldn't",
'y',
'you',
"you'd",
"you'll",
'your',
"you're",
'yours',
'yourself',
'yourselves',
"you've"]
```

```
[51]: import string
      string.punctuation
```

```
[51]: '!"#$%&\'()*+,-./:;<=>?@[\\]^_`{|}~'
```

```
[52]: transform_text('I like krishnaik lectures on ML,DL,NLP.What about you?')
```

```
-----
NameError                                Traceback (most recent call last)
Cell In[52], line 1
----> 1 transform_text('I like krishnaik lectures on ML,DL,NLP.What about you?')

Cell In[49], line 20, in transform_text(text)
     17 y.clear()
     19 for i in text:
--> 20     y.append(ps.stem(i))
     22 return " ".join(y)

NameError: name 'ps' is not defined
```

```
[53]: df1['text']
```

```
[53]: 0      Go until jurong point, crazy.. Available only ...
      1      Ok lar... Joking wif u oni...
      2      Free entry in 2 a wkly comp to win FA Cup fina...
      3      U dun say so early hor... U c already then say...
      4      Nah I don't think he goes to usf, he lives aro...

      ...
5567      This is the 2nd time we have tried 2 contact u...
5568      Will I_ b going to esplanade fr home?
```

```

5569     Pity, * was in mood for that. So...any other s...
5570     The guy did some bitching but I acted like i'd...
5571                                     Rofl. Its true to its name
Name: text, Length: 5169, dtype: object

```

```
[54]: df1['text'][100]
```

```
[54]: 'Okay name ur price as long as its legal! Wen can I pick them up? Y u ave x ams
xx'
```

```
[55]: from nltk.stem.porter import PorterStemmer
ps=PorterStemmer()
ps.stem('singing')
```

```
[55]: 'sing'
```

```
[56]: df1['transformed_text']=df1['text'].apply(transform_text)
```

```

C:\Users\Chinnu\AppData\Local\Temp\ipykernel_3408\3658173627.py:1:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

```

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user\_guide/indexing.html#returning-a-view-versus-a-copy
df1['transformed_text']=df1['text'].apply(transform_text)

```

```
[57]: df1
```

```

[57]:      target      text \
0      0  Go until jurong point, crazy.. Available only ...
1      0      Ok lar... Joking wif u oni...
2      1  Free entry in 2 a wkly comp to win FA Cup fina...
3      0  U dun say so early hor... U c already then say...
4      0  Nah I don't think he goes to usf, he lives aro...
...    ...
5567    1  This is the 2nd time we have tried 2 contact u...
5568    0      Will I_ b going to esplanade fr home?
5569    0  Pity, * was in mood for that. So...any other s...
5570    0  The guy did some bitching but I acted like i'd...
5571    0      Rofl. Its true to its name

      num_characters  num_words  num_sentences  \
0                  111        24             2
1                   29         8             2
2                  155        37             2
3                   49        13             1
4                   61        15             1

```

...	...	...	...
5567	161	35	4
5568	37	9	1
5569	57	15	2
5570	125	27	1
5571	26	7	2

		transformed_text
0	go jurong point crazi avail bugi n great world...	
1		ok lar joke wif u oni
2	free entri 2 wkli comp win fa cup final tkt 21...	
3		u dun say earli hor u c already say
4		nah think goe usf live around though
...		...
5567	2nd time tri 2 contact u pound prize 2 claim e...	
5568		b go esplanad fr home
5569		piti mood suggest
5570	guy bitch act like interest buy someth els nex...	
5571		rofl true name

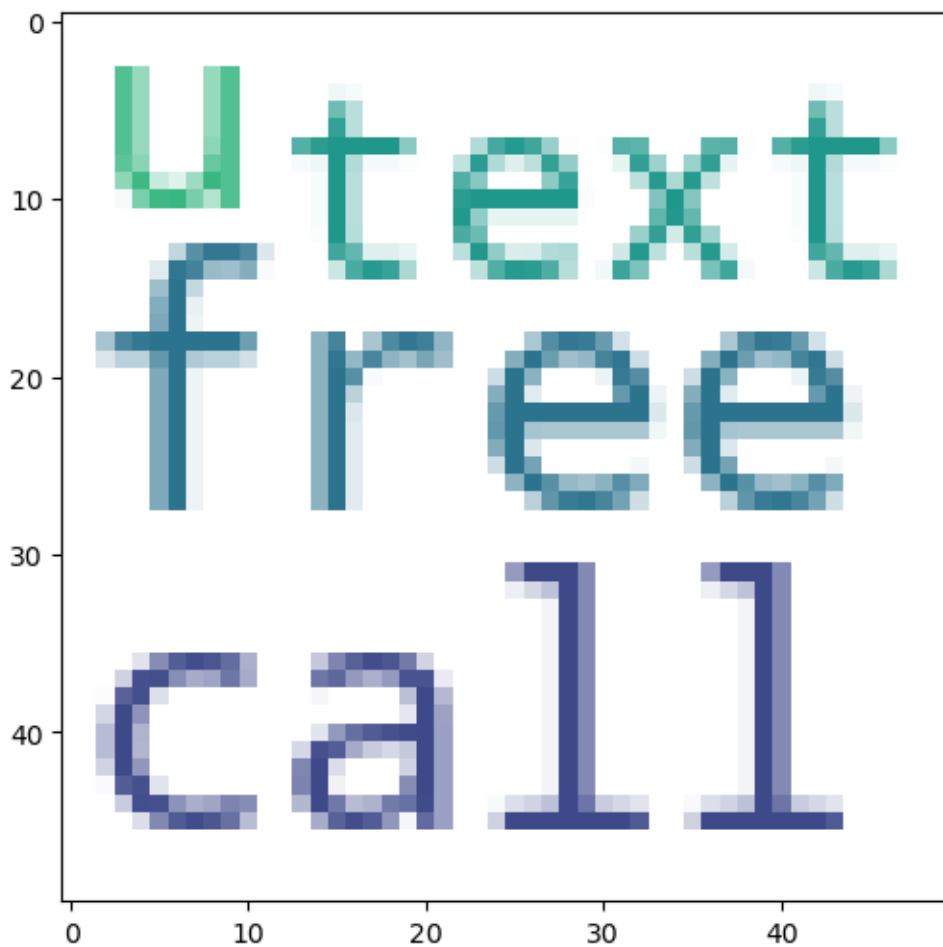
[5169 rows x 6 columns]

```
[58]: from wordcloud import WordCloud
wc=WordCloud(width=50,height=50,min_font_size=10,background_color="white")
```

```
[59]: spam_wc=wc.generate(df1[df1['target']==1]['transformed_text'].str.cat(sep=" "))
```

```
[60]: plt.figure(figsize=(12,6))
plt.imshow(spam_wc)
```

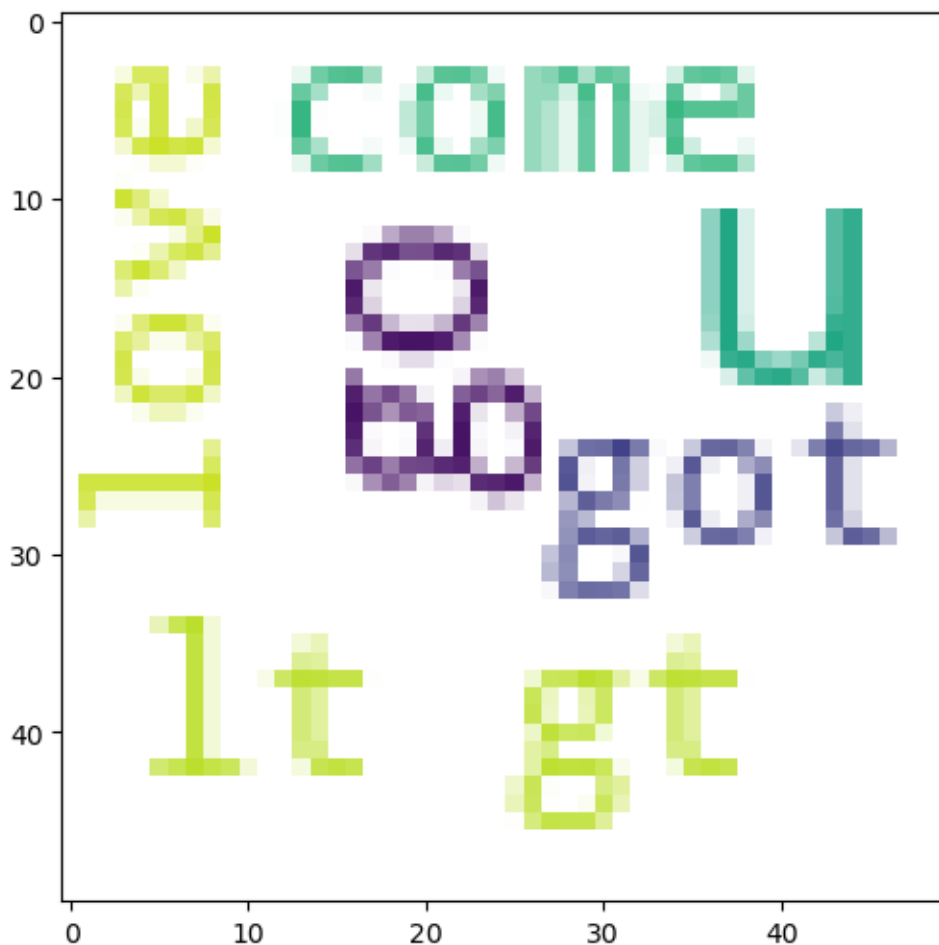
```
[60]: <matplotlib.image.AxesImage at 0x1cb94544560>
```



```
[61]: ham_wc=wc.generate(df1[df1['target']==0]['transformed_text'].str.cat(sep=" "))
```

```
[62]: plt.figure(figsize=(12,6))  
plt.imshow(ham_wc)
```

```
[62]: <matplotlib.image.AxesImage at 0x1cb9d1b0770>
```



```
[63]: df1.head()
```

```
[63]:
```

	target		text	num_characters	\
0	0		Go until jurong point, crazy.. Available only ...	111	
1	0		Ok lar... Joking wif u oni...	29	
2	1		Free entry in 2 a wkly comp to win FA Cup fina...	155	
3	0		U dun say so early hor... U c already then say...	49	
4	0		Nah I don't think he goes to usf, he lives aro...	61	

	num_words	num_sentences	transformed_text
0	24	2	go jurong point crazi avail bugi n great world...
1	8	2	ok lar joke wif u oni
2	37	2	free entri 2 wkli comp win fa cup final tkt 21...
3	13	1	u dun say earli hor u c already say
4	15	1	nah think goe usf live around though

```
[64]: spam_corpus=[]
      for msg in df1[df1['target']==1]['transformed_text'].tolist():
          for word in msg.split():
              spam_corpus.append(word)
```

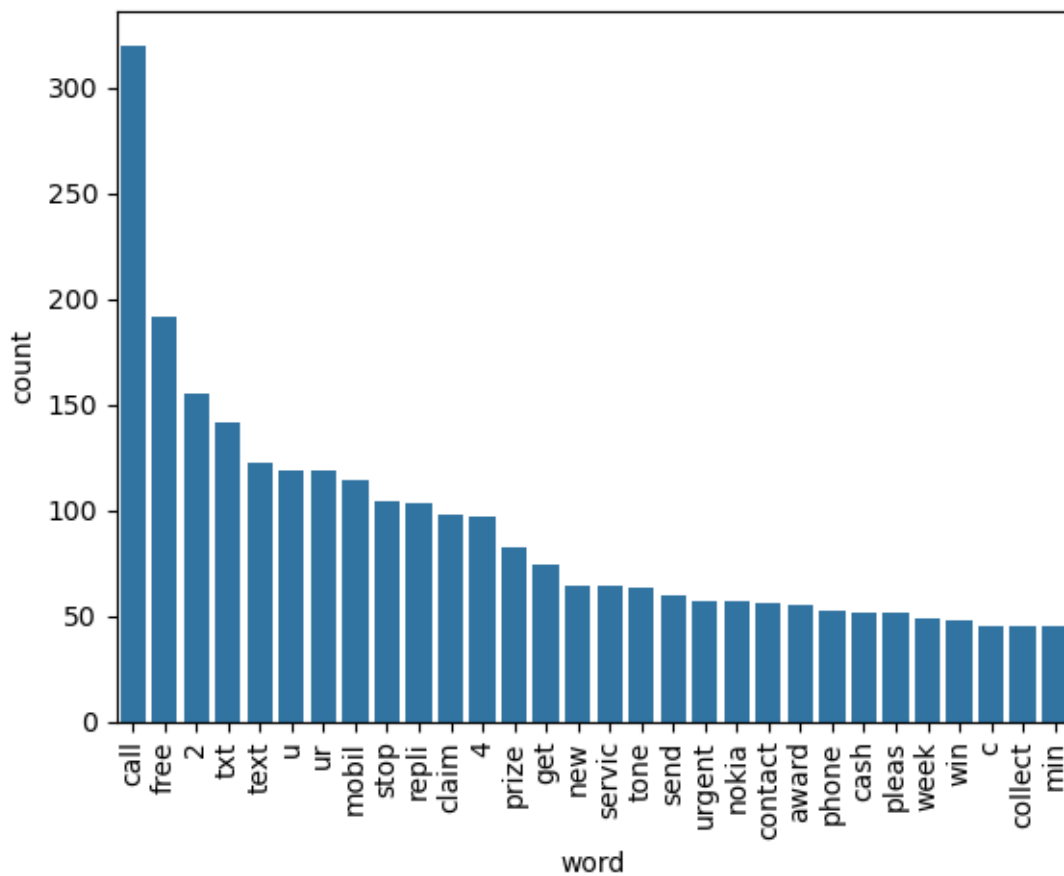
```
[65]: len(spam_corpus)
```

```
[65]: 9939
```

```
[66]: import warnings
      warnings.filterwarnings('ignore')
```

```
[67]: from collections import Counter
      # Prepare data
      common_words = Counter(spam_corpus).most_common(30)
      df_common = pd.DataFrame(common_words, columns=['word', 'count'])

      # Plot with keyword args
      sns.barplot(x='word', y='count', data=df_common)
      plt.xticks(rotation='vertical')
      plt.show()
```

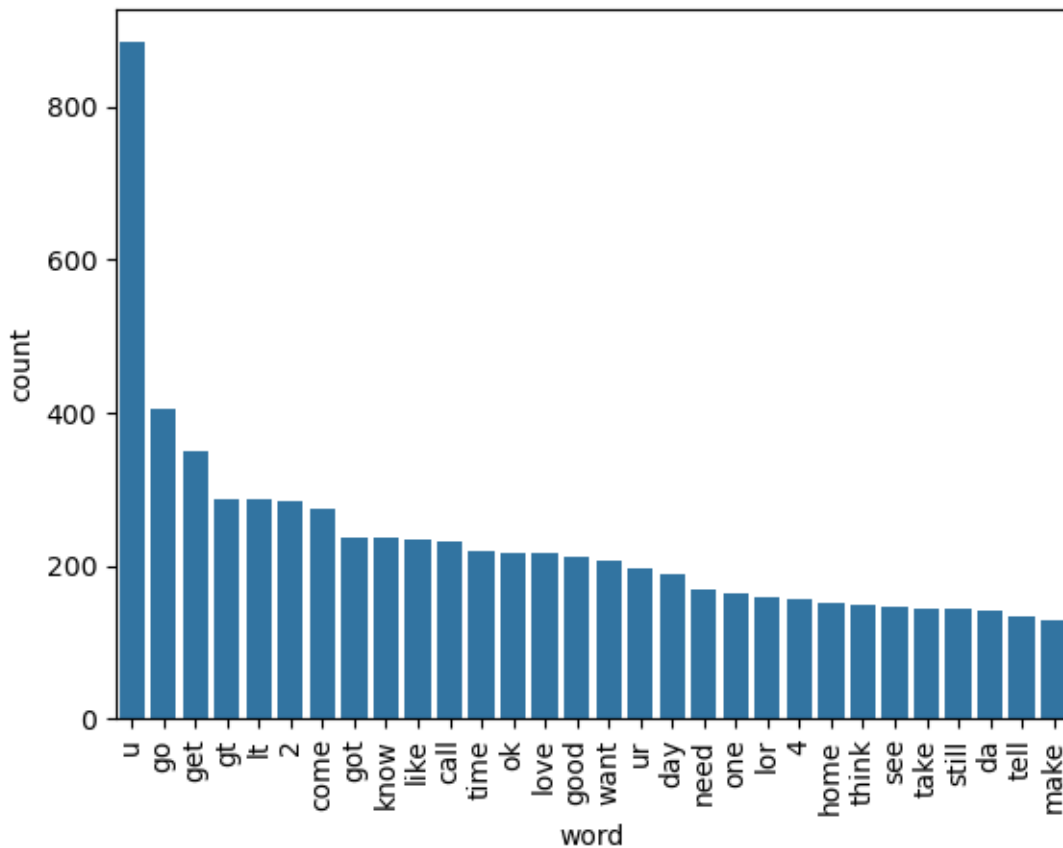


```
[68]: ham_corpus=[]  
      for msg in df1[df1['target']==0]['transformed_text'].tolist():  
          for word in msg.split():  
              ham_corpus.append(word)
```

```
[69]: len(ham_corpus)
```

```
[69]: 35404
```

```
[70]: from collections import Counter  
      # Prepare data  
      common_words = Counter(ham_corpus).most_common(30)  
      df_common = pd.DataFrame(common_words, columns=['word', 'count'])  
  
      # Plot with keyword args  
      sns.barplot(x='word', y='count', data=df_common)  
      plt.xticks(rotation='vertical')  
      plt.show()
```



Model Building

```
[71]: df1.head()
```

```
[71]:
```

	target	text	num_characters	\
0	0	Go until jurong point, crazy.. Available only ...	111	
1	0	Ok lar... Joking wif u oni...	29	
2	1	Free entry in 2 a wkly comp to win FA Cup fina...	155	
3	0	U dun say so early hor... U c already then say...	49	
4	0	Nah I don't think he goes to usf, he lives aro...	61	

	num_words	num_sentences	transformed_text
0	24	2	go jurong point crazi avail bugi n great world...
1	8	2	ok lar joke wif u oni
2	37	2	free entri 2 wkli comp win fa cup final tkt 21...
3	13	1	u dun say earli hor u c already say
4	15	1	nah think goe usf live around though

```
[72]: from sklearn.feature_extraction.text import CountVectorizer,TfidfVectorizer
cv=CountVectorizer()
tfidf=TfidfVectorizer(max_features=3000)
```

```
[74]: #X=cv.fit_transform(df1['transformed_text']).toarray()
X=tfidf.fit_transform(df1['transformed_text']).toarray()
```

```
[75]: X.shape
```

```
[75]: (5169, 3000)
```

```
[76]: Y=df1['target'].values
```

```
[77]: Y
```

```
[77]: array([0, 0, 1, ..., 0, 0, 0])
```

```
[78]: from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB,MultinomialNB,BernoulliNB
from sklearn.metrics import accuracy_score,confusion_matrix,precision_score
```

```
[79]: X_train,X_test,Y_train,Y_test=train_test_split(X,Y,test_size=0.2,random_state=2)
```

```
[80]: gnb=GaussianNB()
mnf=MultinomialNB()
bnb=BernoulliNB()
```

```
[81]: gnb.fit(X_train,Y_train)
      Y_pred1=gnb.predict(X_test)
      print(accuracy_score(Y_test,Y_pred1))
      print(confusion_matrix(Y_test,Y_pred1))
      print(precision_score(Y_test,Y_pred1))
```

```
0.8694390715667312
[[788 108]
 [ 27 111]]
0.5068493150684932
```

```
[82]: mnb.fit(X_train,Y_train)
      Y_pred1=mnb.predict(X_test)
      print(accuracy_score(Y_test,Y_pred1))
      print(confusion_matrix(Y_test,Y_pred1))
      print(precision_score(Y_test,Y_pred1))
```

```
0.9709864603481625
[[896   0]
 [ 30 108]]
1.0
```

```
[83]: bnb.fit(X_train,Y_train)
      Y_pred1=bnb.predict(X_test)
      print(accuracy_score(Y_test,Y_pred1))
      print(confusion_matrix(Y_test,Y_pred1))
      print(precision_score(Y_test,Y_pred1))
```

```
0.9835589941972921
[[895   1]
 [ 16 122]]
0.991869918699187
```

```
[84]: from sklearn.linear_model import LogisticRegression
      from sklearn.svm import SVC
      from sklearn.naive_bayes import MultinomialNB
      from sklearn.tree import DecisionTreeClassifier
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.ensemble import RandomForestClassifier
      from sklearn.ensemble import AdaBoostClassifier
      from sklearn.ensemble import BaggingClassifier
      from sklearn.ensemble import ExtraTreesClassifier
      from sklearn.ensemble import GradientBoostingClassifier
      from xgboost import XGBClassifier
```

```
[85]: svc=SVC(kernel='sigmoid',gamma=1.0)
      knc=KNeighborsClassifier()
      mnb=MultinomialNB()
      dtc=DecisionTreeClassifier(max_depth=5)
```

```

lrc=LogisticRegression(solver='liblinear',penalty='l1')
rfc=RandomForestClassifier(n_estimators=50,random_state=2)
abc=AdaBoostClassifier(n_estimators=50,random_state=2)
bc=BaggingClassifier(n_estimators=50,random_state=2)
etc=ExtraTreesClassifier(n_estimators=50,random_state=2)
gbdt=GradientBoostingClassifier(n_estimators=50,random_state=2)
xgb=XGBClassifier(n_estimators=50,random_state=2)

```

```

[86]: clfs={
    'SVC':svc,
    'KN':knc,
    'NB':mnbc,
    'DT':dtc,
    'LR':lrc,
    'RF':rfc,
    'AdaBoost':abc,
    'BgC':bc,
    'ETC':etc,
    'GBDT':gbdt,
    'xgb':xgb
}

```

```

[87]: def train_classifier(clf,X_train,X_test,Y_train,Y_test):
    clf.fit(X_train,Y_train)
    y_pred=clf.predict(X_test)
    accuracy=accuracy_score(Y_test,y_pred)
    precision=precision_score(Y_test,y_pred)
    return accuracy,precision

```

```

[88]: train_classifier(svc,X_train,X_test,Y_train,Y_test)

```

```

[88]: (0.9758220502901354, 0.9747899159663865)

```

```

[89]: accuracy_scores=[]
precision_scores=[]

for name,clf in clfs.items():
    ↵
    ↪current_accuracy,current_precision=train_classifier(clf,X_train,X_test,Y_train,Y_test)
    print("For",name)
    print("Accuracy -",current_accuracy)
    print("Precision -",current_precision)
    accuracy_scores.append(current_accuracy)
    precision_scores.append(current_precision)

```

For SVC

Accuracy - 0.9758220502901354

```

Precision - 0.9747899159663865
For KN
Accuracy - 0.9052224371373307
Precision - 1.0
For NB
Accuracy - 0.9709864603481625
Precision - 1.0
For DT
Accuracy - 0.9274661508704062
Precision - 0.8118811881188119
For LR
Accuracy - 0.9584139264990329
Precision - 0.9702970297029703
For RF
Accuracy - 0.9758220502901354
Precision - 0.9829059829059829
For AdaBoost
Accuracy - 0.960348162475822
Precision - 0.9292035398230089
For BgC
Accuracy - 0.9584139264990329
Precision - 0.8682170542635659
For ETC
Accuracy - 0.9748549323017408
Precision - 0.9745762711864406
For GBDT
Accuracy - 0.9468085106382979
Precision - 0.9191919191919192
For xgb
Accuracy - 0.9671179883945842
Precision - 0.9482758620689655

```

```

[90]: performance_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy':
    ↳accuracy_scores,'Precision':precision_scores}).
    ↳sort_values('Precision',ascending=False)

```

```

[91]: performance_df

```

```

[91]:
   Algorithm  Accuracy  Precision
1         KN    0.905222    1.000000
2         NB    0.970986    1.000000
5         RF    0.975822    0.982906
0         SVC    0.975822    0.974790
8         ETC    0.974855    0.974576
4         LR    0.958414    0.970297
10        xgb    0.967118    0.948276
6   AdaBoost    0.960348    0.929204

```

```

9      GBDT  0.946809  0.919192
7      BgC   0.958414  0.868217
3      DT    0.927466  0.811881

```

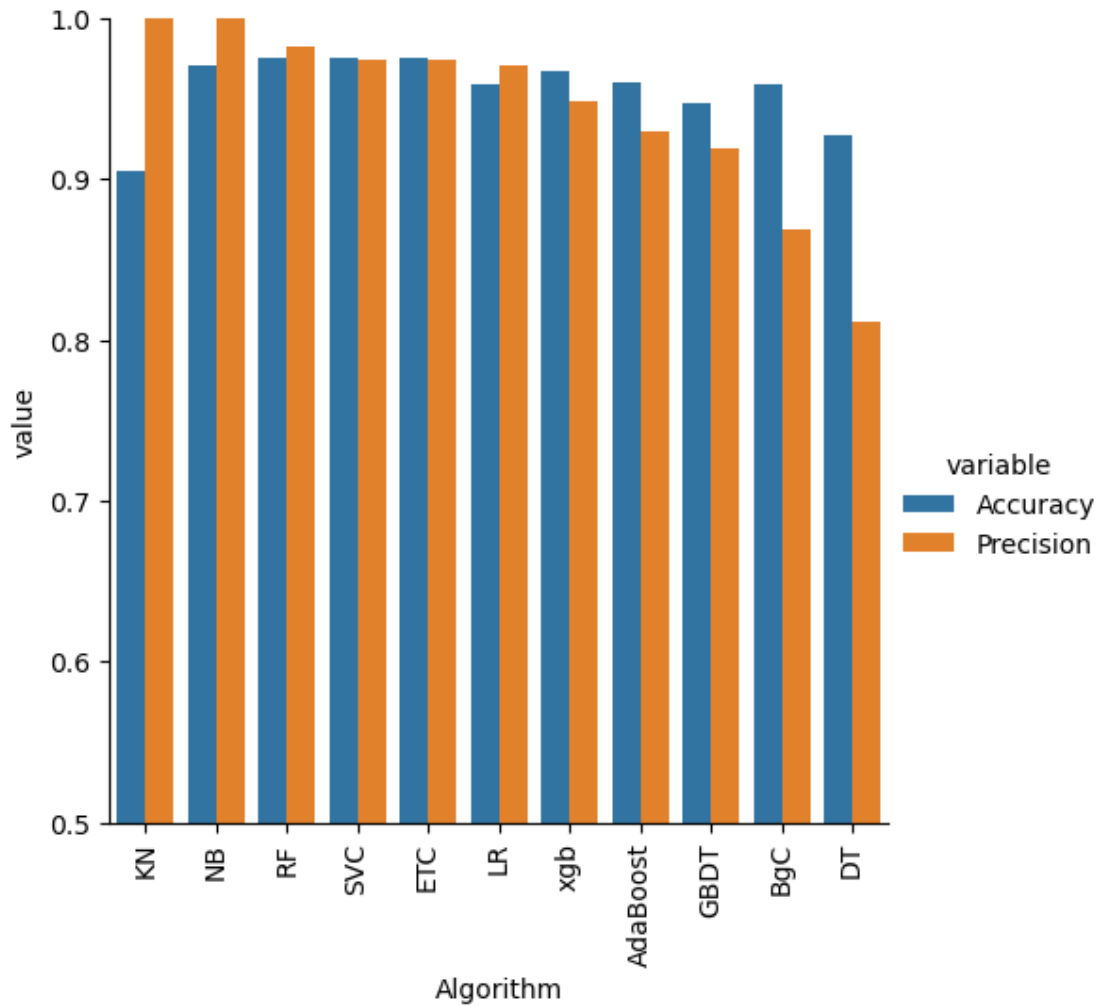
```
[92]: performance_df1 = pd.melt(performance_df, id_vars = "Algorithm")
```

```
[93]: performance_df1
```

```
[93]:
```

	Algorithm	variable	value
0	KN	Accuracy	0.905222
1	NB	Accuracy	0.970986
2	RF	Accuracy	0.975822
3	SVC	Accuracy	0.975822
4	ETC	Accuracy	0.974855
5	LR	Accuracy	0.958414
6	xgb	Accuracy	0.967118
7	AdaBoost	Accuracy	0.960348
8	GBDT	Accuracy	0.946809
9	BgC	Accuracy	0.958414
10	DT	Accuracy	0.927466
11	KN	Precision	1.000000
12	NB	Precision	1.000000
13	RF	Precision	0.982906
14	SVC	Precision	0.974790
15	ETC	Precision	0.974576
16	LR	Precision	0.970297
17	xgb	Precision	0.948276
18	AdaBoost	Precision	0.929204
19	GBDT	Precision	0.919192
20	BgC	Precision	0.868217
21	DT	Precision	0.811881

```
[94]: sns.
      ↪catplot(x='Algorithm',y='value',hue='variable',data=performance_df1,kind='bar',height=5)
plt.ylim(0.5,1.0)
plt.xticks(rotation='vertical')
plt.show()
```



```
[95]: temp_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy_max_ft_3000':
    ↳accuracy_scores,'Precision_max_ft_3000':precision_scores}).
    ↳sort_values('Precision_max_ft_3000',ascending=False)
```

```
[96]: temp_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy_scaling':
    ↳accuracy_scores,'Precision_scaling':precision_scores}).
    ↳sort_values('Precision_scaling',ascending=False)
```

```
[97]: new_df = performance_df.merge(temp_df,on='Algorithm')
```

```
[98]: new_df_scaled = new_df.merge(temp_df,on='Algorithm')
```

```
[99]: temp_df = pd.DataFrame({'Algorithm':clfs.keys(),'Accuracy_num_chars':
    ↳accuracy_scores,'Precision_num_chars':precision_scores}).
    ↳sort_values('Precision_num_chars',ascending=False)
```

```
[100]: new_df_scaled.merge(temp_df,on='Algorithm')
```

```
[100]:
```

	Algorithm	Accuracy	Precision	Accuracy_scaling_x	Precision_scaling_x	\
0	KN	0.905222	1.000000	0.905222	1.000000	
1	NB	0.970986	1.000000	0.970986	1.000000	
2	RF	0.975822	0.982906	0.975822	0.982906	
3	SVC	0.975822	0.974790	0.975822	0.974790	
4	ETC	0.974855	0.974576	0.974855	0.974576	
5	LR	0.958414	0.970297	0.958414	0.970297	
6	xgb	0.967118	0.948276	0.967118	0.948276	
7	AdaBoost	0.960348	0.929204	0.960348	0.929204	
8	GBDT	0.946809	0.919192	0.946809	0.919192	
9	BgC	0.958414	0.868217	0.958414	0.868217	
10	DT	0.927466	0.811881	0.927466	0.811881	

	Accuracy_scaling_y	Precision_scaling_y	Accuracy_num_chars	\
0	0.905222	1.000000	0.905222	
1	0.970986	1.000000	0.970986	
2	0.975822	0.982906	0.975822	
3	0.975822	0.974790	0.975822	
4	0.974855	0.974576	0.974855	
5	0.958414	0.970297	0.958414	
6	0.967118	0.948276	0.967118	
7	0.960348	0.929204	0.960348	
8	0.946809	0.919192	0.946809	
9	0.958414	0.868217	0.958414	
10	0.927466	0.811881	0.927466	

	Precision_num_chars
0	1.000000
1	1.000000
2	0.982906
3	0.974790
4	0.974576
5	0.970297
6	0.948276
7	0.929204
8	0.919192
9	0.868217
10	0.811881

```
[101]: # Voting Classifier
svc = SVC(kernel='sigmoid', gamma=1.0,probability=True)
mnb = MultinomialNB()
etc = ExtraTreesClassifier(n_estimators=50, random_state=2)

from sklearn.ensemble import VotingClassifier
```

```
[102]: voting = VotingClassifier(estimators=[('svm', svc), ('nb', mnb), ('et',  
↳ etc)], voting='soft')
```

```
[103]: voting.fit(X_train, Y_train)
```

```
[103]: VotingClassifier(estimators=[('svm',  
SVC(gamma=1.0, kernel='sigmoid',  
probability=True)),  
( 'nb', MultinomialNB()),  
( 'et',  
ExtraTreesClassifier(n_estimators=50,  
random_state=2))],  
voting='soft')
```

```
[104]: y_pred = voting.predict(X_test)  
print("Accuracy", accuracy_score(Y_test, y_pred))  
print("Precision", precision_score(Y_test, y_pred))
```

```
Accuracy 0.9816247582205029  
Precision 0.9917355371900827
```

```
[105]: # Applying stacking  
estimators=[('svm', svc), ('nb', mnb), ('et', etc)]  
final_estimator=RandomForestClassifier()
```

```
[106]: from sklearn.ensemble import StackingClassifier
```

```
[107]: clf = StackingClassifier(estimators=estimators, final_estimator=final_estimator)
```

```
[108]: clf.fit(X_train, Y_train)  
y_pred = clf.predict(X_test)  
print("Accuracy", accuracy_score(Y_test, y_pred))  
print("Precision", precision_score(Y_test, y_pred))
```

```
Accuracy 0.9806576402321083  
Precision 0.946969696969697
```

```
[109]: import pickle  
  
pickle.dump(tfidf, open('vectorizer.pkl', 'wb'))  
pickle.dump(mnb, open('model.pkl', 'wb'))
```

```
[ ]:
```